

# Jenkins: Cẩm nang Chuyên sâu về Kiến trúc, Tự động hóa và Vận hành cho Kỹ sư DevOps

## 1. Giới thiệu và Vai trò Cốt lõi của Jenkins trong Kỷ nguyên DevOps

### 1.1. Lịch sử và Vị thế trong Hệ sinh thái CI/CD

Trong bối cảnh phát triển phần mềm hiện đại, tốc độ và độ tin cậy được xem là hai trụ cột quyết định sự thành bại của một sản phẩm công nghệ. Kỷ nguyên của việc tích hợp thủ công và chu kỳ phát hành kéo dài hàng tháng đã lùi vào dĩ vãng, nhường chỗ cho sự lên ngôi của tự động hóa—một quy trình liền mạch kết nối việc viết mã (coding), kiểm thử (testing) và triển khai (deploying).<sup>1</sup> Tại trung tâm của cuộc cách mạng này là Jenkins, một máy chủ tự động hóa mã nguồn mở (open-source automation server) đóng vai trò như một "nhạc trưởng" điều phối mọi hoạt động trong quy trình Tích hợp liên tục (Continuous Integration - CI) và Chuyển giao liên tục (Continuous Delivery - CD).<sup>1</sup>

Ban đầu được phát triển với tên gọi Hudson, Jenkins đã trải qua một quá trình phát triển dài và tách ra (fork) thành dự án riêng biệt sau những tranh chấp về quản lý thương hiệu với Oracle. Mặc dù hiện nay có sự xuất hiện của nhiều công cụ CI/CD hiện đại như GitHub Actions, GitLab CI hay CircleCI, Jenkins vẫn giữ vững vị thế là công cụ phổ biến nhất, được sử dụng bởi hàng nghìn đội ngũ phát triển trên toàn cầu, từ các công ty khởi nghiệp đến các tập đoàn lớn.<sup>2</sup> Sự trường tồn của Jenkins đến từ khả năng tùy biến vô hạn thông qua hệ thống plugin khổng lồ, hỗ trợ hầu hết các công cụ xây dựng (build tools) như Maven, Gradle, và các nền tảng container hóa như Docker và Kubernetes.<sup>2</sup>

Đối với một thực tập sinh (Intern) bước chân vào lĩnh vực DevOps, việc nắm vững Jenkins không chỉ dừng lại ở việc học cách sử dụng một công cụ cụ thể. Đó là quá trình thấu hiểu sâu sắc về tư duy tự động hóa, cách quản lý quy trình xây dựng phức tạp và giải quyết các vấn đề về tích hợp hệ thống.<sup>4</sup> Báo cáo này sẽ cung cấp một cái nhìn toàn diện, từ kiến trúc nền tảng đến các kỹ thuật vận hành nâng cao, giúp kỹ sư tương lai làm chủ công nghệ này.

### 1.2. Khái niệm CI/CD và Vai trò Trung tâm của Jenkins

Để hiểu rõ cách Jenkins vận hành, trước tiên cần phân tích cơ chế của một quy trình CI/CD tiêu chuẩn—được coi là huyết mạch của văn hóa DevOps.

**Tích hợp liên tục (Continuous Integration - CI)** là thực hành trong đó các lập trình viên thường xuyên hợp nhất (merge) các thay đổi mã nguồn vào một kho lưu trữ chia sẻ (shared

repository). Mỗi khi có một thay đổi được đẩy lên (push), Jenkins sẽ tự động kích hoạt một quy trình xây dựng và kiểm thử. Mục tiêu của CI là phát hiện lỗi tích hợp càng sớm càng tốt, giảm thiểu sự phức tạp khi gỡ lỗi (debugging) ở giai đoạn sau.<sup>1</sup> Jenkins hoạt động như một hệ thống giám sát không mệt mỏi; ngay khi nhận diện được commit mới từ GitHub, Bitbucket hay GitLab, nó sẽ tải mã nguồn về, biên dịch và chạy các bài kiểm tra tự động.<sup>1</sup>

**Chuyển giao và Triển khai liên tục (Continuous Delivery/Deployment - CD)** mở rộng quy trình CI bằng cách tự động hóa việc đưa mã nguồn đã được kiểm thử sang các môi trường tiếp theo như môi trường dàn dựng (staging) hoặc sản xuất (production). Trong khi Continuous Delivery đảm bảo rằng phần mềm luôn ở trạng thái sẵn sàng để triển khai với sự can thiệp tối thiểu của con người, thì Continuous Deployment đi xa hơn bằng việc tự động hóa hoàn toàn việc phát hành sản phẩm ra thị trường nếu tất cả các bài kiểm tra đều vượt qua.<sup>1</sup>

Jenkins đóng vai trò là "người điều phối" (orchestrator) kết nối các giai đoạn rời rạc này. Nó không chỉ đơn thuần là chạy các tập lệnh; nó quản lý trạng thái của quy trình, lưu trữ các tạo phẩm (artifacts) sau khi build, và cung cấp phản hồi (feedback) tức thời thông qua log và thông báo.<sup>5</sup> Mặc dù Jenkins không phải là một giải pháp CI gốc của Kubernetes (Kubernetes native) hay container-native, kiến trúc linh hoạt của nó cho phép tích hợp sâu rộng với các công nghệ này để đóng gói và kiểm thử phần mềm trong các môi trường cách ly.<sup>2</sup>

## 2. Kiến trúc Hệ thống Phân tán: Controller và Agent

Sức mạnh thực sự của Jenkins nằm ở khả năng mở rộng thông qua kiến trúc phân tán (distributed architecture). Hiểu rõ mối quan hệ giữa Controller và Agent là chìa khóa để thiết kế một hệ thống CI/CD có hiệu năng cao và khả năng chịu lỗi tốt.

### 2.1. Jenkins Controller (Bộ điều khiển trung tâm)

Trong các phiên bản trước đây, thành phần này được gọi là "Master". Tuy nhiên, để phù hợp với ngôn ngữ bao trùm (inclusive language), cộng đồng Jenkins đã chuyển sang thuật ngữ "Controller". Controller là bộ não của hệ thống, chịu trách nhiệm quản lý toàn bộ cấu hình, điều phối luồng công việc và tương tác với người dùng.<sup>2</sup>

Chức năng nhiệm vụ của Controller:

Controller lưu trữ cấu hình của toàn bộ hệ thống và các job cụ thể. Nó phục vụ giao diện người dùng (Web UI) thông qua giao thức HTTP/HTTPS, cho phép quản trị viên thiết lập pipeline, quản lý người dùng và xem kết quả build.<sup>7</sup> Khi một job được kích hoạt, Controller sẽ chịu trách nhiệm phân tích cú pháp của pipeline, xác định tài nguyên cần thiết và lên lịch (schedule) cho các tác vụ.<sup>6</sup>

Một điểm quan trọng cần lưu ý trong vận hành là Controller không nên trực tiếp thực thi các tác vụ xây dựng (build tasks). Việc thực thi build trên Controller tiêu tốn tài nguyên CPU và RAM, có thể làm chậm giao diện quản lý hoặc thậm chí gây treo hệ thống. Đây là lý do tại sao các chuyên gia DevOps luôn khuyến nghị thiết lập số lượng "executors" trên Controller về 0.<sup>8</sup>

Điều này đảm bảo Controller chỉ tập trung vào nhiệm vụ quản lý và điều phối, chuyển giao toàn bộ gánh nặng xử lý sang các Agent.<sup>7</sup>

## 2.2. Jenkins Agent (Tác nhân thực thi)

Agent (trước đây gọi là Slave) là các máy tính hoặc container riêng biệt kết nối với Controller để thực hiện các công việc cụ thể. Trong ngữ cảnh của Jenkins, thuật ngữ "Node" là khái niệm bao trùm chỉ bất kỳ hệ thống nào có khả năng thực thi pipeline, bao gồm cả Agent và Controller (nếu được cấu hình).<sup>2</sup>

Agent là nơi công việc thực sự diễn ra. Một hệ thống Jenkins có thể có hàng trăm Agent chạy trên các hệ điều hành khác nhau (Linux, Windows, macOS) và kiến trúc phần cứng khác nhau (x86, ARM). Điều này cho phép kiểm thử ứng dụng đa nền tảng một cách dễ dàng. Ví dụ, một pipeline có thể yêu cầu build ứng dụng iOS trên một Agent macOS, đồng thời chạy kiểm thử backend trên một Agent Linux.<sup>8</sup>

Jenkins phân loại các tác vụ thành hai loại:

- **Flyweight Task:** Các tác vụ nhẹ chạy trên Controller để điều phối công việc, không chiếm dụng executor trên Agent.<sup>6</sup>
- **Heavyweight Task:** Các tác vụ nặng (như biên dịch code, chạy test) chiếm dụng một executor slot trên Agent. Trong môi trường autoscaling, quy tắc chung là cấu hình 1 executor cho mỗi Agent để tối ưu hóa hiệu năng.<sup>6</sup>

## 2.3. Các Giao thức Kết nối và Chiến lược Mở rộng

Để Controller giao tiếp và phân phối công việc cho Agent, Jenkins hỗ trợ hai phương thức kết nối chính, mỗi phương thức có ưu nhược điểm riêng phù hợp với từng ngữ cảnh hạ tầng.<sup>6</sup>

### SSH Connector (Secure Shell)

Đây là phương thức kết nối phổ biến và ổn định nhất, đặc biệt trong môi trường Linux/Unix. Trong mô hình này, Controller đóng vai trò là SSH Client và chủ động khởi tạo kết nối đến Agent (SSH Server). Quá trình này hoàn toàn trong suốt và tận dụng cơ chế bảo mật của SSH.<sup>7</sup>

Ưu điểm của SSH là sự đơn giản trong quản trị. Yêu cầu duy nhất là Agent phải được cài đặt Java và Public Key của Controller phải nằm trong file `authorized_keys` của Agent. Khi kết nối được thiết lập, Controller sẽ tự động sao chép file `agent.jar` sang Agent và khởi chạy tiến trình.<sup>6</sup> Đây là lựa chọn lý tưởng cho các Agent là máy chủ vật lý hoặc máy ảo (VM) cố định trong mạng nội bộ.

### JNLP (Java Network Launch Protocol) / Inbound Agent

Phương thức này, hiện nay thường được gọi là "Inbound Agent", đảo ngược chiều kết nối so với SSH. Thay vì Controller kết nối đến Agent, Agent sẽ chủ động kết nối ra (outbound) tới

Controller thông qua giao thức TCP hoặc WebSocket.<sup>9</sup>

Mô hình JNLP đặc biệt hữu ích trong các trường hợp sau:

- Agent nằm sau tường lửa (Firewall) hoặc NAT mà Controller không thể truy cập trực tiếp.
- Agent chạy trong môi trường container động (như Kubernetes Pods hoặc Docker Containers), nơi địa chỉ IP thay đổi liên tục và không cố định.<sup>9</sup>
- Yêu cầu bảo mật không cho phép mở cổng SSH (port 22) trên Agent.<sup>9</sup>

Bảng so sánh chi tiết dưới đây giúp kỹ sư DevOps lựa chọn phương thức phù hợp:

| Đặc điểm               | SSH Connector                  | JNLP / Inbound Agent                             |
|------------------------|--------------------------------|--------------------------------------------------|
| Hướng khởi tạo kết nối | Controller $\rightarrow$ Agent | Agent $\rightarrow$ Controller                   |
| Yêu cầu mạng           | Mở port 22 trên Agent          | Mở port JNLP (ví dụ: 50000) trên Controller      |
| Độ phức tạp cấu hình   | Thấp (Chỉ cần SSH Key)         | Trung bình (Cần tải agent.jar và chạy lệnh Java) |
| Môi trường phù hợp     | Máy chủ vật lý, VM cố định     | Container, Cloud động, Mạng bị hạn chế NAT       |
| Tính ổn định           | Rất cao                        | Phụ thuộc vào kết nối mạng từ phía Agent         |

### 3. Cài đặt và Triển khai Jenkins: Hướng dẫn Kỹ thuật

Việc triển khai Jenkins đòi hỏi sự hiểu biết về môi trường hệ điều hành và các phụ thuộc phần mềm. Hai phương pháp phổ biến nhất là cài đặt trực tiếp trên Linux và chạy thông qua Docker. Đối với một intern DevOps, việc làm chủ Docker là kỹ năng bắt buộc.<sup>10</sup>

#### 3.1. Triển khai Jenkins trên Docker (Khuyến nghị)

Sử dụng Docker giúp đóng gói môi trường Jenkins, dễ dàng sao lưu, nâng cấp và tránh các xung đột thư viện với hệ điều hành máy chủ (Host OS). Tuy nhiên, một thách thức đặt ra là làm sao để Jenkins (chạy trong container) có thể gọi lệnh Docker để build image

(Docker-in-Docker).

Quy trình cài đặt tiêu chuẩn bao gồm các bước sau <sup>10</sup>:

#### Bước 1: Tạo Docker Network

Tạo một bridge network để các container liên quan (Jenkins Controller và Docker Daemon) có thể giao tiếp với nhau bằng tên định danh (DNS resolution).

Bash

```
docker network create jenkins
```

#### Bước 2: Chạy Container Docker-in-Docker (DinD)

Đây là thành phần cung cấp Docker Daemon cho Jenkins sử dụng. Việc sử dụng cờ `--privileged` là bắt buộc để container này có quyền truy cập sâu vào kernel của host, cho phép nó quản lý các container con.

Bash

```
docker run --name jenkins-docker --rm --detach \
  --privileged --network jenkins --network-alias docker \
  --env DOCKER_TLS_CERTDIR=/certs \
  --volume jenkins-docker-certs:/certs/client \
  --volume jenkins-data:/var/jenkins_home \
  --publish 2376:2376 \
  docker:dind
```

*Lưu ý:* Việc map volume `jenkins-docker-certs` giúp chia sẻ chứng chỉ bảo mật giữa client (Jenkins) và server (DinD).

#### Bước 3: Chạy Container Jenkins Controller

Sử dụng image chính thức `jenkins/jenkins:its` để đảm bảo tính ổn định (Long Term Support).

Bash

```
docker run --name jenkins-blueocean --restart=on-failure --detach \
```

```
--network jenkins --env DOCKER_HOST=tcp://docker:2376 \
--env DOCKER_CERT_PATH=/certs/client --env DOCKER_TLS_VERIFY=1 \
--publish 8080:8080 --publish 50000:50000 \
--volume jenkins-data:/var/jenkins_home \
--volume jenkins-docker-certs:/certs/client:ro \
jenkins/jenkins:lts-jdk17
```

*Giải thích tham số:*

- 8080: Cổng truy cập giao diện web.
- 50000: Cổng dành cho JNLP Agent kết nối.
- DOCKER\_HOST: Chỉ định địa chỉ của Docker Daemon (container jenkins-docker ở trên).

## 3.2. Cài đặt trên Linux (CentOS/Ubuntu)

Nếu không sử dụng Docker, việc cài đặt trực tiếp trên Linux yêu cầu cài đặt Java Runtime Environment (JRE) trước tiên. Jenkins yêu cầu Java 11 hoặc Java 17.<sup>13</sup>

Trên các hệ thống Red Hat/CentOS:

1. Cài đặt Java: `sudo yum install java-17-openjdk -y`
2. Thêm kho lưu trữ Jenkins (Repository):  
Bash  
`sudo wget -O /etc/yum.repos.d/jenkins.repo  
https://pkg.jenkins.io/redhat-stable/jenkins.repo  
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key`
3. Cài đặt Jenkins: `sudo yum install jenkins -y`
4. Khởi động dịch vụ: `sudo systemctl start jenkins`.<sup>14</sup>

## 3.3. Thiết lập Ban đầu (Initial Setup Wizard)

Sau khi cài đặt, Jenkins sẽ kích hoạt chế độ bảo mật mặc định. Người dùng truy cập `http://localhost:8080` sẽ gặp màn hình "Unlock Jenkins".<sup>15</sup>

Mở khóa Jenkins:

Mật khẩu quản trị ban đầu (initialAdminPassword) được sinh ra ngẫu nhiên và lưu tại:

- Đường dẫn file: `/var/jenkins_home/secrets/initialAdminPassword`.<sup>11</sup>
- Trong log Docker: Sử dụng lệnh `docker logs jenkins-blueocean` và tìm chuỗi ký tự nằm giữa hai hàng dấu sao.<sup>16</sup>

Lựa chọn Plugin:

Trình hướng dẫn sẽ đưa ra hai tùy chọn:

1. **Install suggested plugins:** Tự động cài đặt bộ plugin phổ biến nhất (Git, Pipeline, Folders, v.v.). Đây là lựa chọn khuyên dùng cho người mới bắt đầu để đảm bảo có đủ

công cụ cơ bản.<sup>17</sup>

2. **Select plugins to install:** Cho phép tùy chỉnh danh sách plugin. Chỉ nên dùng khi bạn có yêu cầu đặc thù và muốn tối giản hệ thống.<sup>17</sup>

Sau bước này, bạn sẽ tạo tài khoản Admin đầu tiên và cấu hình URL cho Jenkins instance. Việc cấu hình đúng URL rất quan trọng để các liên kết trong email thông báo và webhook hoạt động chính xác.<sup>18</sup>

## 4. Pipeline as Code: Ngôn ngữ của Tự động hóa

Sự chuyển dịch từ cấu hình giao diện (UI-based) sang cấu hình bằng mã (Code-based) là bước tiến lớn nhất của Jenkins 2.0. "Pipeline as Code" cho phép định nghĩa toàn bộ quy trình CI/CD trong một tệp văn bản có tên là Jenkinsfile, được lưu trữ và kiểm soát phiên bản (version control) cùng với mã nguồn ứng dụng.<sup>19</sup>

### 4.1. Declarative Pipeline so với Scripted Pipeline

Jenkins hỗ trợ hai cú pháp viết pipeline, đều dựa trên ngôn ngữ Groovy, nhưng phục vụ các mục đích và đối tượng khác nhau. Việc phân biệt rõ ràng hai loại này giúp kỹ sư chọn giải pháp phù hợp.<sup>21</sup>

Declarative Pipeline (Cú pháp Khai báo):

Được giới thiệu sau này nhằm đơn giản hóa việc viết pipeline. Nó cung cấp một cấu trúc chặt chẽ, dễ đọc và dễ bảo trì.

- **Đặc điểm:** Bắt đầu bằng khối pipeline {...}. Cú pháp hạn chế việc sử dụng code Groovy phức tạp, tập trung vào việc khai báo "làm cái gì" (what) hơn là "làm như thế nào" (how).<sup>22</sup>
- **Ưu điểm:** Hỗ trợ kiểm tra cú pháp (linting) tốt hơn, tự động khởi động lại từ giai đoạn thất bại (Restart from Stage), và tích hợp trực quan hoàn hảo với giao diện Blue Ocean.<sup>23</sup>
- **Khuyến nghị:** Đây là tiêu chuẩn hiện tại cho hầu hết các dự án và là lựa chọn bắt buộc cho người mới bắt đầu.<sup>24</sup>

Scripted Pipeline (Cú pháp Kịch bản):

Là cú pháp nguyên bản của Jenkins Pipeline, dựa hoàn toàn vào Groovy.

- **Đặc điểm:** Bắt đầu bằng khối node {...}. Cho phép sử dụng toàn bộ sức mạnh của ngôn ngữ Groovy, bao gồm các vòng lặp phức tạp, cấu trúc điều kiện nâng cao và xử lý ngoại lệ.<sup>21</sup>
- **Ưu điểm:** Cực kỳ linh hoạt, phù hợp cho các luồng logic nghiệp vụ rất phức tạp mà Declarative không thể đáp ứng.
- **Nhược điểm:** Khó đọc, khó bảo trì, và dễ gặp lỗi cú pháp mà chỉ phát hiện được khi chạy (runtime).<sup>21</sup>

**Bảng so sánh Cú pháp:**

| Tiêu chí      | Declarative Pipeline            | Scripted Pipeline      |
|---------------|---------------------------------|------------------------|
| Khởi bắt đầu  | pipeline { ... }                | node { ... }           |
| Độ linh hoạt  | Trung bình (Cấu trúc định sẵn)  | Rất cao (Groovy thuần) |
| Độ dễ đọc     | Cao, cấu trúc rõ ràng           | Thấp, code phức tạp    |
| Kiểm tra lỗi  | Kiểm tra cú pháp trước khi chạy | Phát hiện lỗi khi chạy |
| Restart Stage | Hỗ trợ                          | Không hỗ trợ native    |

## 4.2. Giải phẫu một Declarative Pipeline

Để viết được một Jenkinsfile chuẩn, cần nắm vững các thành phần cốt lõi sau <sup>20</sup>:

- Agent (Tác nhân):** Chỉ định nơi pipeline hoặc một stage cụ thể sẽ được thực thi.
  - agent any: Chạy trên bất kỳ agent nào đang rảnh.
  - agent none: Không gán agent ở cấp toàn cục, bắt buộc từng stage phải tự định nghĩa agent riêng.
  - agent { docker { image 'node:16' } }: Yêu cầu Jenkins khởi tạo một Docker container từ image chỉ định và chạy các lệnh bên trong đó. Đây là tính năng mạnh mẽ giúp đảm bảo môi trường build sạch sẽ và nhất quán.<sup>26</sup>
- Stages (Các giai đoạn):** Là tập hợp các bước thực hiện. Một pipeline thường bao gồm các stage tiêu chuẩn như "Build", "Test", "Deploy". Các stage này sẽ hiển thị trực quan trên giao diện người dùng, giúp dễ dàng theo dõi tiến độ.<sup>25</sup>
- Steps (Các bước):** Đơn vị thực thi nhỏ nhất trong một stage. Chứa các lệnh thực tế như sh (shell script trên Linux), bat (batch script trên Windows), git (lấy mã nguồn), junit (báo cáo test).<sup>25</sup>
- Post (Hậu xử lý):** Định nghĩa các hành động xảy ra sau khi pipeline hoặc stage kết thúc. Đây là nơi lý tưởng để gửi thông báo hoặc dọn dẹp tài nguyên.<sup>28</sup>
  - always: Luôn chạy bất kể kết quả build.
  - success: Chỉ chạy khi build thành công.
  - failure: Chỉ chạy khi build thất bại.
  - unstable: Chạy khi có lỗi test nhưng không nghiêm trọng đến mức fail build.

## Ví dụ cấu trúc Declarative Pipeline:

Groovy

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Đang biên dịch mã nguồn...'
        sh 'make build'
      }
    }
    stage('Test') {
      steps {
        echo 'Đang chạy kiểm thử...'
        sh 'make test'
      }
    }
  }
  post {
    failure {
      echo 'Build thất bại! Gửi thông báo...'
    }
  }
}
```

### 4.3. Quản lý Biến Môi trường và Bảo mật

Trong thực tế, pipeline cần tương tác với nhiều tham số động và thông tin nhạy cảm (mật khẩu, API key).

**Biến Môi trường (Environment Variables):**

Jenkins cung cấp sẵn một loạt biến toàn cục như BUILD\_NUMBER, JOB\_NAME, GIT\_COMMIT. Người dùng có thể truy cập chúng thông qua đối tượng env (ví dụ: env.BUILD\_ID) trong Groovy hoặc \$BUILD\_ID trong Shell script. Ngoài ra, khối environment {...} cho phép định nghĩa các biến mới áp dụng cho toàn bộ pipeline hoặc riêng từng stage.<sup>29</sup>

**Cơ chế Script Security (Sandbox):**

Vì lý do bảo mật, các đoạn script Groovy trong pipeline chạy trong môi trường Sandbox bị hạn chế quyền. Nếu pipeline cố gắng sử dụng các phương thức Java nội bộ chưa được phê duyệt

(ví dụ: java.io.File), Jenkins sẽ chặn lại và báo lỗi Scripts not permitted to use method. Quản trị viên phải truy cập trang "In-process Script Approval" để phê duyệt thủ công các phương thức này.<sup>31</sup> Đây là cơ chế bảo vệ quan trọng để ngăn chặn mã độc thực thi trên server.

## 5. Tích hợp Quản lý Mã nguồn (SCM) và Chiến lược

### Kích hoạt

Kết nối Jenkins với hệ thống quản lý mã nguồn (Source Control Management) như Git là bước đầu tiên của mọi quy trình CI.

#### 5.1. Xác thực và Bảo mật với Git

Jenkins hỗ trợ nhiều phương thức xác thực để truy cập vào các kho lưu trữ riêng tư (Private Repository).<sup>33</sup>

1. **SSH Keys:** Đây là phương thức an toàn và phổ biến nhất. Kỹ sư tạo một cặp khóa SSH (Public/Private). Public Key được thêm vào cấu hình "Deploy Keys" trên GitHub/GitLab, trong khi Private Key được lưu trữ an toàn trong Jenkins Credentials dưới loại "SSH Username with private key".<sup>34</sup>
2. **Personal Access Token (PAT):** Đối với kết nối qua HTTPS, việc sử dụng mật khẩu tài khoản Git thông thường không còn được khuyến khích (và GitHub đã loại bỏ). Thay vào đó, cần sử dụng PAT và lưu trong Jenkins Credentials dưới dạng "Username with password".<sup>35</sup>

Trong Pipeline, lệnh checkout scm được sử dụng để tự động lấy mã nguồn dựa trên cấu hình của job, giúp đơn giản hóa script và tránh hard-code URL của repository.<sup>30</sup>

#### 5.2. Webhook so với Polling SCM: Cuộc chiến Hiệu năng

Làm thế nào để Jenkins biết khi nào cần chạy build? Có hai cơ chế chính, và sự lựa chọn ảnh hưởng lớn đến hiệu năng hệ thống.<sup>36</sup>

Polling SCM (Cơ chế Hỏi định kỳ):

Jenkins được cấu hình để định kỳ (ví dụ: mỗi 5 phút) "hỏi" Git server xem có commit mới không.

- **Ưu điểm:** Dễ cấu hình, hoạt động tốt ngay cả khi Jenkins nằm trong mạng nội bộ không có IP công khai.
- **Nhược điểm:** Tiêu tốn tài nguyên mạng và CPU không cần thiết. Có độ trễ giữa lúc code được push và lúc build bắt đầu. Không phù hợp cho quy mô lớn.

Webhook (Cơ chế Đẩy sự kiện):

Đây là phương pháp hiện đại và tối ưu nhất. Git server (GitHub/GitLab) sẽ chủ động gửi một yêu cầu HTTP POST (Payload) tới Jenkins ngay khi có sự kiện (Push, Merge Request).

- **Ưu điểm:** Phản hồi tức thời (Real-time), tiết kiệm tài nguyên vì Jenkins chỉ hoạt động khi có việc.

- **Nhược điểm:** Yêu cầu Jenkins phải có địa chỉ IP công khai hoặc cấu hình mạng để Git server có thể kết nối tới. Trong môi trường phát triển local, có thể dùng các công cụ như Ngrok để giả lập đường dẫn.<sup>38</sup>

**Khuyến nghị:** Luôn ưu tiên sử dụng Webhook cho môi trường sản xuất để đạt hiệu quả tối đa.<sup>37</sup>

## 6. Hệ sinh thái Plugin và Các Công cụ Hỗ trợ

Sức mạnh của Jenkins nằm ở khả năng mở rộng. Dưới đây là phân tích chi tiết về các plugin và công cụ không thể thiếu cho một DevOps Engineer vào năm 2025.<sup>39</sup>

### 6.1. Blue Ocean: Trải nghiệm Người dùng Thế hệ mới

Giao diện cổ điển của Jenkins thường bị chê là lỗi thời và khó sử dụng. Blue Ocean ra đời để giải quyết vấn đề này với giao diện trực quan, hiện đại, tập trung vào trải nghiệm Pipeline.<sup>41</sup>

- **Trực quan hóa Pipeline:** Blue Ocean hiển thị pipeline dưới dạng biểu đồ các stage nối tiếp nhau. Khi build thất bại, người dùng có thể dễ dàng xác định chính xác stage nào gặp lỗi và xem log của riêng stage đó, thay vì phải tìm kiếm trong hàng nghìn dòng log hỗn loạn.<sup>42</sup>
- **Pipeline Editor:** Cung cấp công cụ soạn thảo trực quan, cho phép kéo thả để tạo pipeline mà không cần viết code Groovy thủ công, rất hữu ích cho người mới bắt đầu.<sup>42</sup>

### 6.2. Credentials Binding: Quản lý Bí mật An toàn

Không bao giờ được phép lưu trữ mật khẩu, API token trực tiếp trong Jenkinsfile (hard-coding). Plugin Credentials Binding cho phép inject (tiêm) các thông tin nhạy cảm này vào pipeline dưới dạng biến môi trường. Điều quan trọng là Jenkins sẽ tự động "mask" (che dấu) các giá trị này trong log console (hiển thị thành \*\*\*\*), đảm bảo an toàn thông tin.<sup>43</sup>

### 6.3. Docker Pipeline và Tích hợp Container

Plugin này cho phép pipeline tương tác sâu với Docker Daemon.

- **Agent Docker:** Cho phép pipeline chạy bên trong container, đảm bảo môi trường build luôn đồng nhất, không phụ thuộc vào các công cụ cài sẵn trên máy chủ vật lý.<sup>26</sup>
- **Docker Build & Publish:** Hỗ trợ các lệnh như `docker.build()` và `docker.image().push()` để tự động đóng gói ứng dụng và đẩy lên Docker Registry (như Docker Hub hoặc ECR).<sup>45</sup>

### 6.4. SonarQube Scanner: Kiểm soát Chất lượng Mã nguồn

Tự động hóa không chỉ là build nhanh, mà còn là đảm bảo chất lượng. Plugin SonarQube Scanner tích hợp việc phân tích mã nguồn tĩnh (Static Analysis) vào pipeline. Sử dụng block `withSonarQubeEnv`, Jenkins tự động cấu hình thông tin kết nối tới SonarQube Server, chạy

phân tích và có thể chặn quy trình deployment nếu code không đạt chuẩn (Quality Gate failed).<sup>46</sup>

## 6.5. Slack Notification: Giao tiếp Thời gian thực

Trong môi trường làm việc nhóm, việc nhận thông báo tức thời về trạng thái build là rất quan trọng. Plugin Slack cho phép Jenkins gửi tin nhắn vào các kênh chat của dự án. Kỹ sư có thể tùy chỉnh màu sắc tin nhắn (Xanh cho thành công, Đỏ cho thất bại) và nội dung chi tiết (người commit, thời gian chạy), giúp đội ngũ phản ứng nhanh với các sự cố.<sup>48</sup>

## 7. Các Thực hành Tốt nhất (Best Practices) cho Vận hành Ổn định

Để vận hành Jenkins hiệu quả và tránh các "thảm họa" kỹ thuật, intern DevOps cần tuân thủ nghiêm ngặt các nguyên tắc sau.<sup>24</sup>

### 7.1. Bảo mật Hệ thống

- **Phân quyền (RBAC):** Tuyệt đối không sử dụng chế độ bảo mật lỏng lẻo. Sử dụng "Matrix Authorization Strategy" hoặc "Role-based Authorization Strategy" để cấp quyền chi tiết. Ví dụ: Developer chỉ có quyền xem và build job, trong khi DevOps Lead mới có quyền cấu hình hệ thống.<sup>52</sup>
- **Chạy dưới quyền User hạn chế:** Service Jenkins trên Linux không bao giờ được chạy dưới quyền root. Phải tạo một user riêng (thường là jenkins) với quyền hạn tối thiểu cần thiết để giảm thiểu thiệt hại nếu hệ thống bị tấn công.<sup>12</sup>

### 7.2. Tối ưu hóa Pipeline

- **Dọn dẹp Workspace (Clean Workspace):** Ổ cứng đầy là nguyên nhân hàng đầu gây lỗi build. Luôn sử dụng lệnh cleanWs() trong phần post { always {...} } để xóa các file tạm sau khi build xong.<sup>53</sup>
- **Đặt giới hạn thời gian (Timeout):** Một job bị treo có thể chiếm dụng executor vô thời hạn, gây tắc nghẽn toàn bộ hệ thống. Luôn thiết lập timeout cho mỗi stage hoặc toàn bộ pipeline (ví dụ: 1 giờ) để Jenkins tự động hủy job nếu chạy quá lâu.<sup>25</sup>
- **Sử dụng Shared Libraries:** Khi nhiều pipeline có logic giống nhau, đừng sao chép code (Don't Repeat Yourself - DRY). Hãy đưa logic đó vào "Shared Libraries" và gọi lại trong các Jenkinsfile. Điều này giúp việc bảo trì và cập nhật logic trở nên dễ dàng và đồng bộ.<sup>54</sup>

### 7.3. Chiến lược Sao lưu (Backup)

Jenkins lưu trữ toàn bộ cấu hình trong thư mục JENKINS\_HOME dưới dạng các file XML. Việc sao lưu thư mục này là bắt buộc. Sử dụng plugin "ThinBackup" để lên lịch sao lưu định kỳ cấu hình (không cần sao lưu lịch sử build quá cũ để tiết kiệm dung lượng). Nên đồng bộ các bản

backup này lên Cloud Storage (S3, Google Cloud Storage) để đảm bảo an toàn dữ liệu.<sup>24</sup>

## 8. Khắc phục Sự cố (Troubleshooting) và Bảo trì

Khả năng chẩn đoán lỗi là kỹ năng phân biệt giữa một intern và một kỹ sư chuyên nghiệp. Dưới đây là phân tích sâu về các vấn đề thường gặp.<sup>53</sup>

### 8.1. Vấn đề Bộ nhớ (OutOfMemoryError - OOM)

Jenkins được viết bằng Java và chạy trên JVM. Nếu Heap Size được cấp phát không đủ, hệ thống sẽ gặp lỗi `java.lang.OutOfMemoryError: Java heap space` và bị treo.

- **Chẩn đoán:** Kiểm tra log hệ thống hoặc sử dụng công cụ giám sát như VisualVM. Nếu thấy biểu đồ sử dụng bộ nhớ liên tục chạm đỉnh (max out), đó là dấu hiệu thiếu RAM.
- **Giải pháp:** Tăng Heap Size thông qua tham số khởi động `-Xmx`. Ví dụ, trong file `/etc/default/jenkins` hoặc biến môi trường `JAVA_OPTS` của Docker, thiết lập `-Xmx4g` (cấp tối đa 4GB RAM). Nếu vấn đề vẫn tái diễn dù đã tăng RAM, cần xem xét việc phân tích Heap Dump để tìm lỗi rò rỉ bộ nhớ (memory leak) do plugin gây ra.<sup>56</sup>

### 8.2. Lỗi Quyền Truy cập (Permission Denied)

Đây là lỗi kinh điển khi Jenkins cố gắng thực thi các lệnh shell hoặc truy cập file.

- **Nguyên nhân:** User jenkins không có quyền ghi vào thư mục workspace hoặc không có quyền truy cập vào socket của Docker (`/var/run/docker.sock`).
- **Giải pháp:** Đảm bảo user jenkins là chủ sở hữu (owner) của thư mục làm việc. Đối với Docker, cần thêm user jenkins vào nhóm docker (`sudo usermod -aG docker jenkins`). Tuy nhiên, cần lưu ý rằng việc này tương đương với việc cấp quyền root cho Jenkins, vì vậy cần cân nhắc kỹ về bảo mật hoặc sử dụng giải pháp Rootless Docker.<sup>53</sup>

### 8.3. Sự cố Kết nối Agent (Offline Agents)

Agent bị ngắt kết nối sẽ làm gián đoạn quy trình CI/CD.

- **Nguyên nhân:** Có thể do đầy ổ cứng trên Agent (Jenkins có cơ chế tự bảo vệ, ngắt kết nối agent nếu dung lượng đĩa trống  $< 1\text{GB}$ ), lệch thời gian hệ thống (Clock skew) quá lớn giữa Controller và Agent, hoặc mạng không ổn định.
- **Giải pháp:** Kiểm tra log trên Agent (`agent.log`), giải phóng dung lượng đĩa, và cài đặt đồng bộ thời gian (NTP) trên tất cả các node trong hệ thống.<sup>55</sup>

## 9. Kết luận và Định hướng Phát triển

Jenkins, với lịch sử phát triển lâu đời và hệ sinh thái khổng lồ, vẫn là một kỹ năng không thể thiếu đối với bất kỳ kỹ sư DevOps nào. Dù cho xu hướng chuyển dịch sang các nền tảng CI/CD dựa trên đám mây (Cloud-based) như GitHub Actions đang diễn ra mạnh mẽ, Jenkins vẫn giữ vai trò trụ cột trong các hệ thống Doanh nghiệp (Enterprise) yêu cầu tính bảo mật cao, khả

năng tùy biến sâu và vận hành trên hạ tầng riêng (On-premise).<sup>4</sup>

Đối với một thực tập sinh, lộ trình phát triển tiếp theo nên tập trung vào việc kết hợp Jenkins với Kubernetes để triển khai các **Ephemeral Agents** (Agent dùng một lần). Mô hình này cho phép tạo ra các môi trường build động, tự hủy sau khi dùng xong, giải quyết triệt để vấn đề về quản lý tài nguyên và sự không nhất quán của môi trường.<sup>52</sup> Làm chủ Jenkins không chỉ là làm chủ một công cụ, mà là làm chủ tư duy hệ thống—nền tảng vững chắc cho sự nghiệp DevOps bền vững.

## Nguồn trích dẫn

1. How Does Jenkins Work Within a CI/CD Pipeline?, truy cập vào tháng 1 14, 2026, <https://medium.com/@haroldfinch01/how-does-jenkins-work-within-a-ci-cd-pipeline-0dba97f9c49c>
2. What Is Jenkins and How Does it Work? Intro and Tutorial - Codefresh, truy cập vào tháng 1 14, 2026, <https://codefresh.io/learn/jenkins/>
3. What's the best way for a DevOps fresher to learn Jenkins? Any recommended resources or learning paths? - Reddit, truy cập vào tháng 1 14, 2026, [https://www.reddit.com/r/devops/comments/1fwmb0w/whats\\_the\\_best\\_way\\_for\\_a\\_devops\\_fresher\\_to\\_learn/](https://www.reddit.com/r/devops/comments/1fwmb0w/whats_the_best_way_for_a_devops_fresher_to_learn/)
4. jenkins first steps : r/devops - Reddit, truy cập vào tháng 1 14, 2026, [https://www.reddit.com/r/devops/comments/1if4qle/jenkins\\_first\\_steps/](https://www.reddit.com/r/devops/comments/1if4qle/jenkins_first_steps/)
5. CICD with Jenkins - BrowserStack, truy cập vào tháng 1 14, 2026, <https://www.browserstack.com/guide/ci-cd-with-jenkins>
6. [BEGINNER] Confused about the very definition of an Agent : r/jenkinsci - Reddit, truy cập vào tháng 1 14, 2026, [https://www.reddit.com/r/jenkinsci/comments/zfb0pp/beginner\\_confused\\_about\\_the\\_very\\_definition\\_of\\_an/](https://www.reddit.com/r/jenkinsci/comments/zfb0pp/beginner_confused_about_the_very_definition_of_an/)
7. Architecting for Scale - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/scaling/architecting-for-scale/>
8. Using Jenkins agents, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/using/using-agents/>
9. Day 28: Jenkins Controller-Agent Architecture via JNLP | by Dipali Naygaonkar | Medium, truy cập vào tháng 1 14, 2026, <https://medium.com/@dipali26N/day-28-jenkins-controller-agent-architecture-via-jnlp-c9f2602b3c6b>
10. How To Install Jenkins On Docker, truy cập vào tháng 1 14, 2026, <https://www.youtube.com/watch?v=ncmx8-ywbFM>
11. Install Jenkins on Linux step-by-step Using Docker Compose | by Suresh yadav - Medium, truy cập vào tháng 1 14, 2026, <https://medium.com/cypik/step-by-step-install-jenkins-on-amazon-linux-7c8750185e7f>
12. Docker - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/installing/docker/>
13. How do you install Jenkins?. Jenkins is the backbone of modern... | by Harold

- Finch | Nov, 2025, truy cập vào tháng 1 14, 2026,  
<https://medium.com/@haroldfinch01/how-do-you-install-jenkins-f67c155e56db>
14. How to Install and Set Up Jenkins on Your Personal Linux Machine (RHEL/CentOS/Fedora), truy cập vào tháng 1 14, 2026,  
<https://medium.com/@eodenyire/how-to-install-and-set-up-jenkins-on-your-personal-linux-machine-rhel-centos-fedora-7b421d83516d>
  15. Windows - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/installing/windows/>
  16. How to "Unlock Jenkins"? - Stack Overflow, truy cập vào tháng 1 14, 2026,  
<https://stackoverflow.com/questions/35960883/how-to-unlock-jenkins>
  17. Linux - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/installing/linux/>
  18. Other Systems - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/installing/other/>
  19. Pipeline as Code with Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/solutions/pipeline/>
  20. Pipeline - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/pipeline/>
  21. Understanding the Differences Between Jenkins Scripted and Declarative Pipeline: A Comprehensive Guide with Real-World Examples - Praveen Dandu, truy cập vào tháng 1 14, 2026,  
<https://praveendandu24.medium.com/understanding-the-differences-between-jenkins-scripted-and-declarative-pipeline-a-comprehensive-960826e26c2>
  22. Declarative vs Scripted Pipeline - Key differences | DS Stream Data Science, truy cập vào tháng 1 14, 2026,  
<https://www.dsstream.com/post/declarative-vs-scripted-pipeline-key-differences>
  23. Jenkins scripted pipeline or declarative pipeline - Stack Overflow, truy cập vào tháng 1 14, 2026,  
<https://stackoverflow.com/questions/43484979/jenkins-scripted-pipeline-or-declarative-pipeline>
  24. Jenkins Best Practices every Developer must know in 2025 - BrowserStack, truy cập vào tháng 1 14, 2026,  
<https://www.browserstack.com/guide/jenkins-best-practices-every-developer-must-know>
  25. Pipeline Syntax - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/pipeline/syntax/>
  26. Defining execution environments - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/pipeline/tour/agents/>
  27. Pipeline: Basic Steps - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/pipeline/steps/workflow-basic-steps/>
  28. How to send Slack notification after a Jenkins pipeline build failed? - Stack Overflow, truy cập vào tháng 1 14, 2026,  
<https://stackoverflow.com/questions/39140191/how-to-send-slack-notification-after-a-jenkins-pipeline-build-failed>
  29. Understanding Environment Variables in Jenkins: Usage and Examples |

- BrowserStack, truy cập vào tháng 1 14, 2026,  
<https://www.browserstack.com/guide/jenkins-env-variables>
30. Using a Jenkinsfile, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/>
  31. Jenkins: Script not permitted to use method signature | RichHewlett.com, truy cập vào tháng 1 14, 2026,  
<https://richhewlett.com/2020/03/24/jenkins-script-not-permitted-to-use-method-signature/>
  32. Why am I unable to see a method in In-process Script Approval?, truy cập vào tháng 1 14, 2026,  
<https://docs.cloudbees.com/docs/cloudbees-ci-kb/latest/client-and-managed-controllers/why-am-i-unable-to-see-a-method-in-in-process-script-approval>
  33. Using credentials - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/using/using-credentials/>
  34. Setup Jenkins for private GitHub repository | by Maciej Najbar | FAcademy - Medium, truy cập vào tháng 1 14, 2026,  
<https://medium.com/facademy/setup-jenkins-for-private-repository-9060f54eeac9>
  35. Authenticate Jenkins CI for Github private repository - Stack Overflow, truy cập vào tháng 1 14, 2026,  
<https://stackoverflow.com/questions/5212304/authenticate-jenkins-ci-for-github-private-repository>
  36. Jenkins poll vs webhook - github - Stack Overflow, truy cập vào tháng 1 14, 2026,  
<https://stackoverflow.com/questions/43797807/jenkins-poll-vs-webhook>
  37. What Is Poll SCM Jenkins ?, And How To Configure Poll Scm ? - GeeksforGeeks, truy cập vào tháng 1 14, 2026,  
<https://www.geeksforgeeks.org/devops/what-is-poll-scm-jenkins-and-how-to-configure-poll-scm/>
  38. Jenkins Tutorial: Configure (SCM) GitHub Triggers and Git Polling using Ngrok - CloudBees, truy cập vào tháng 1 14, 2026,  
<https://www.cloudbees.com/blog/jenkins-tutorial-configure-scm-github-triggers-and-git-polling-using-ngrok>
  39. Top 5 Jenkins Plugins Every DevOps Engineer Should Use | by Sai Vidyadhar | Medium, truy cập vào tháng 1 14, 2026,  
<https://medium.com/@vidyadhardevops7/top-5-jenkins-plugins-every-devops-engineer-should-use-5400819a9fe0>
  40. Top 20 Jenkins Plugins with Features for 2025 - KnowledgeHut, truy cập vào tháng 1 14, 2026, <https://www.knowledgehut.com/blog/devops/jenkins-plugins>
  41. Create Pipeline using Blue Ocean Graphical Editor - KodeKloud Notes, truy cập vào tháng 1 14, 2026,  
<https://notes.kodekloud.com/docs/Jenkins-For-Beginners/Jenkins-Pipelines/Create-Pipeline-using-Blue-Ocean-Graphical-Editor>
  42. Blue Ocean - Jenkins, truy cập vào tháng 1 14, 2026,  
<https://www.jenkins.io/doc/book/blueocean/>
  43. Git username / password credentials binding - Jenkins, truy cập vào tháng 1 14,

- 2026, <https://www.jenkins.io/blog/2021/07/27/git-credentials-binding-phase-1/>
44. Credentials Binding Plugin - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/pipeline/steps/credentials-binding/>
  45. Using Docker with Pipeline - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/pipeline/docker/>
  46. SonarQube Scanner for Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/pipeline/steps/sonar/>
  47. Adding analysis to a Jenkins job | SonarQube Server 10.7 - Sonar Documentation, truy cập vào tháng 1 14, 2026, <https://docs.sonarsource.com/sonarqube-server/10.7/analyzing-source-code/ci-integration/jenkins-integration/add-analysis-to-job>
  48. Slack Notification Plugin - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/pipeline/steps/slack/>
  49. Notify slack user from Jenkins pipeline when build fails - Stack Overflow, truy cập vào tháng 1 14, 2026, <https://stackoverflow.com/questions/62127056/notify-slack-user-from-jenkins-pipeline-when-build-fails>
  50. Best Practices - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/using/best-practices/>
  51. Essential Jenkins Best Practices for Developers: Streamline Your CI/CD Process - Medium, truy cập vào tháng 1 14, 2026, <https://medium.com/@prateek.malhotra004/essential-jenkins-best-practices-for-developers-streamline-your-ci-cd-process-dc3aa38f9928>
  52. Jenkins Tutorial: From Basic CI To Full CI/CD | - Octopus Deploy, truy cập vào tháng 1 14, 2026, <https://octopus.com/devops/jenkins/jenkins-tutorial/>
  53. Most Common Jenkins Errors and Their Solutions | by Neel Shah - AWS in Plain English, truy cập vào tháng 1 14, 2026, <https://aws.plainenglish.io/most-common-jenkins-errors-and-their-solutions-b9b16435e910>
  54. Pipeline Best Practices - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/pipeline/pipeline-best-practices/>
  55. Jenkins Common Issues And Solutions For Beginners | SLA - Softlogic Systems, truy cập vào tháng 1 14, 2026, <https://www.softlogicsys.in/jenkins-common-issues-and-solutions-for-beginners/>
  56. Diagnosing Errors - Jenkins, truy cập vào tháng 1 14, 2026, <https://www.jenkins.io/doc/book/troubleshooting/diagnosing-errors/>
  57. TOP 10 DevOps Tools in 2025: Based on 300 LinkedIn job posts - Reddit, truy cập vào tháng 1 14, 2026, [https://www.reddit.com/r/devops/comments/1lj93a1/top\\_10\\_devops\\_tools\\_in\\_2025\\_based\\_on\\_300\\_linkedin/](https://www.reddit.com/r/devops/comments/1lj93a1/top_10_devops_tools_in_2025_based_on_300_linkedin/)